SAC - Sprint 2 - Planificador de Tareas

SAC - Sprint 2 - Planificador de Tareas

Contexto

En el Sprint 1 creaste el **wireframe** y el **HTML de tu formulario** para el Planificador de Tareas. Algunos usaron Bootstrap, otros no. Algunos agregaron campos adicionales, otros se quedaron con los mínimos.

En este Sprint 2 vas a dar el salto a la lógica. Hasta ahora solo tienes la estructura visual (HTML). Ahora vas a agregar la programación en JavaScript que hace que el Planificador de Tareas **funcione de verdad**:

- Validar el formulario antes de agregar tareas.
- Crear una clase que gestione las tareas.
- Conectar el formulario con la lógica para agregar tareas.
- Mostrar las tareas en pantalla.
- Eliminar tareas.

Importante: No se te va a decir en qué archivo crear el código, ni cómo organizar tus carpetas, ni qué nombres usar para los IDs. Eso es **decisión tuya**. Lo que se te pide es que cumplas con los objetivos y criterios de evaluación.

Recuerda:

- Bootstrap es **opcional**.
- Las tareas se guardan **solo en memoria** por ahora. La persistencia con localStorage se verá en otros Sprints.
- No se evalúa el diseño visual, sino la funcionalidad.

Lo que Debes Lograr al Final del Sprint 2

Al terminar este Sprint, tu Planificador de Tareas debe poder:

1. **Validar** el formulario antes de agregar tareas.
2. Tener una clase `TaskManager` que gestione una lista de tareas.
3. Agregar tareas desde el formulario a la lista.
4. Mostrar las tareas como tarjetas en pantalla.
5. Eliminar tareas de la lista y de la pantalla.
6. Generar un ID único para cada tarea.
7. Almacenar las tareas en un array en memoria.

Tarea 1: Validación de Entradas en el Formulario de Tareas

Objetivo

Implementar una función en JavaScript que valide los datos del formulario antes de permitir agregar una tarea.

Paso 1: Crear la función `validateTaskForm`

Crea una función llamada `validateTaskForm` que verifique lo siguiente:

Campo	Regla de validación
Nombre	No en blanco y más largo que 8 caracteres.
Descripción	No en blanco y más largo que 15 caracteres.
AsignadoA	No en blanco y más largo que 8 caracteres.
Fecha de Entrega	No en blanco.
Estado	Debe tener un valor seleccionado.

Pistas:

- La función debe devolver `true` si todos los campos son válidos, `false` si alguno es inválido.
- Puedes usar `trim()` para eliminar espacios en blanco al inicio y final.

- Puedes usar `.length` para verificar la longitud de las cadenas.

Paso 2: Mostrar mensajes de error

Cuando un campo sea inválido, debes mostrar un **mensaje de error significativo** al usuario.

Pistas:

- Puedes usar `alert()` para mostrar el mensaje.
- O puedes mostrar el mensaje en un elemento HTML (por ejemplo, un `<div>` de errores).
- El mensaje debe indicar **qué campo** es inválido y **por qué** (por ejemplo: "El nombre debe tener al menos 8 caracteres").

Paso 3: Conectar la validación con el formulario

La función `validateTaskForm` debe ejecutarse **antes** de agregar la tarea. Si la validación falla, la tarea no se agrega y se muestra el mensaje de error.

Pistas:

- La validación se ejecuta en el event listener del formulario (o del botón de envío).
- Si `validateTaskForm()` devuelve `false`, no se debe llamar a `addTask()`.
- Si devuelve `true`, se procede a agregar la tarea.

Criterios de Evaluación - Tarea 1 (Validación)

#	Criterio
1	Todos los campos del formulario se validan al enviar el formulario (Nombre, Descripción, AsignadoA, Fecha de Entrega, Estado).
2	Se muestra un mensaje de error significativo cuando un campo del formulario es inválido.

#	Criterio
3	El código JavaScript está en un archivo por separado y el archivo se incluye en la página HTML sin errores.

Tarea 2: Crear la Clase de Tareas con JavaScript

Objetivo

Implementar una clase JavaScript que represente una tarea y otra clase que gestione la lista de tareas.

Paso 1: Definir la estructura de una tarea

Crea un objeto que represente una tarea con los siguientes campos:

Campo	Tipo	Descripción
`id`	Entero	Identificador único de la tarea
`nombre`	Cadena de texto	Nombre de la tarea
`descripcion`	Cadena de texto	Descripción de la tarea
`asignadoA`	Cadena de texto	Persona responsable de completar la tarea
`fechaDeEntrega`	Fecha	Fecha en que se debe completar la tarea
`estado`	Cadena de texto	Estado de la tarea: `POR HACER`, `EN PROCESO`, `REVISIÓN` o `HECHO`

Pista: Puedes definir este objeto dentro del constructor de la clase `TaskManager` o como una clase separada. Es tu decisión.

Paso 2: Crear la clase `TaskManager`

Crea una clase llamada `TaskManager` que implemente los siguientes métodos:

Método	¿Qué hace?
`getAllTasks()`	Devuelve la lista de **todas** las tareas.
`getTasksWithStatus(status)`	Devuelve una lista de todas las tareas que tengan el estado pasado como argumento.
`addTask(task)`	Agrega una tarea a la lista de tareas existente.
`deleteTask(taskId)`	Elimina una tarea de la lista de tareas.
`updateTask(taskId, status)`	Actualiza el estado de una tarea.
`assignTask(taskId, assignee)`	Actualiza el valor de la persona asignada a una tarea.

Pistas:

- La clase debe tener un **array interno** donde se almacenen las tareas.
- Piensa en cómo generar un **ID único** para cada tarea. Puede ser un contador que se incrementa cada vez que se agrega una tarea.

Criterios de Evaluación - Tarea 2 (Clase TaskManager)

#	Criterio
1	La estructura de la tarea está definida correctamente con todos los campos requeridos.
2	La clase `TaskManager` está definida correctamente.
3	Los métodos de la clase están definidos correctamente.
4	El formato de la tarea es compatible con JSON (puede convertirse a JSON).

Tarea 3: Agregar Tareas de Manera Programada

Objetivo

Implementar la lógica para que, cuando el usuario complete el formulario, la tarea se agregue a la lista y se muestre en pantalla.

Paso 1: Conectar el formulario con `addTask`

Agrega un **event listener** al formulario (o al botón de envío) para que:

1. Se **prevenga** el comportamiento por defecto del formulario (que la página no se recargue).
2. Se **valide** el formulario con `validateTaskForm()`.
3. Si la validación es exitosa, se **lean** los valores de los campos.
4. Se **cree** un objeto de tarea con esos valores.
5. Se **llame** al método `addTask()` de `TaskManager`.

Pistas:

- Usa `addEventListener` con el evento `submit`.
- Usa `event.preventDefault()` para evitar la recarga.
- Usa los IDs de los campos que tú creaste en el Sprint 1.

Paso 2: Crear la tarjeta HTML de la tarea

Dentro de la clase `TaskManager` (o en una función separada), agrega un método que:

1. Reciba un objeto de tarea.
2. Cree un elemento HTML que represente una tarjeta con:
   - Nombre de la tarea.
   - Descripción.
   - AsignadoA.
   - Fecha de entrega.
   - Estado.
3. Devuelva la tarjeta creada.

Pistas:

- Puedes usar `document.createElement()` o `innerHTML`.
- Añade clases CSS para estilizar la tarjeta (opcional).
- Asegúrate de que la tarjeta muestre **toda** la información de la tarea.

Paso 3: Agregar la tarjeta al DOM

Agrega el código para que, después de crear la tarjeta, esta se **agregue al contenedor de tareas** en la página.

Pistas:

- Si tu contenedor de tareas no tiene un ID, **agrégalo** en tu HTML.
- Usa `append()` o `appendChild()` para agregar la tarjeta al contenedor.

Paso 4: Asegurarte de que el ID se incrementa y es único

Cada vez que se agregue una tarea, el ID debe:

1. Ser **único** (no debe repetirse con tareas anteriores).
2. **Incrementarse** automáticamente.

Pista: Usa una variable contador que se incremente cada vez que se agregue una tarea. Puede ser una variable global o una propiedad de la clase `TaskManager`.

Criterios de Evaluación - Tarea 3 (Agregar tareas)

#	Criterio
1	Cuando se agrega una nueva tarea con información válida, los datos se almacenan en un array dentro del archivo JavaScript.
2	La tarea agregada es visible en la lista actual de tareas y muestra la información de la tarea.
3	Al agregar una nueva tarea, el identificador se incrementa y es único.

Tarea 4: Eliminar una Tarea

Objetivo

Implementar la lógica para que el usuario pueda eliminar tareas de la lista.

Paso 1: Agregar un botón de eliminar a cada tarjeta

Modifica el método que crea la tarjeta para que incluya un **botón de eliminar**.

Pistas:

- El botón debe tener un texto claro como "Eliminar".
- Puedes agregar clases CSS para estilizarlo.

Paso 2: Conectar el botón con `deleteTask`

Agrega un **event listener** al botón de eliminar para que:

1. Se **identifique** la tarea a eliminar (por su ID).
2. Se **llame** al método `deleteTask()` de `TaskManager`.
3. Se **elimine** la tarjeta del DOM.

Pistas:

- Usa `addEventListener` con el evento `click`.
- Puedes pasar el ID de la tarea como argumento al event listener.
- Para eliminar la tarjeta del DOM, puedes usar `remove()` en el elemento de la tarjeta.

Paso 3: Eliminar la tarea del array

Asegúrate de que, al llamar a `deleteTask()`:

1. La tarea se elimine del **array interno** de `TaskManager`.
2. La tarjeta se elimine del **DOM**.

Pista: Usa `filter()` para crear un nuevo array sin la tarea eliminada, o `splice()` para eliminarla del array original.

Criterios de Evaluación - Tarea 4 (Eliminar tareas)

#	Criterio
1	El botón de eliminar borra la tarjeta de tarea de la pantalla.
2	El botón de eliminar elimina el objeto de tarea del array de tareas dentro de la clase `TaskManager`.
3	El ID de la tarea encuentra la tarea que el usuario desea eliminar.

Consejos de Buenas Prácticas

Aprovecha este Sprint para **revisar y mejorar** el HTML que hiciste en el Sprint 1. Estos consejos no son obligatorios, pero son buenas prácticas.

Sobre el HTML

#	Consejo
1	Usa **etiquetas semánticas** (`<header>` , `<main>` , `<section>` , `<footer>`) en lugar de solo `<div>`.
2	Cada campo del formulario debe tener una **etiqueta** (`<label>`) asociada con `for`.
3	Usa el atributo `for` del `<label>` para que coincida con el `id` del `<input>`.
4	Agrega el atributo `name` a los campos del formulario.
5	Usa `placeholder` para dar pistas sobre el formato esperado.
6	Usa `required` en los campos que sean obligatorios.

#	Consejo
7	Agrega un `<button type="submit">` para enviar el formulario.
8	Asegúrate de que el formulario tenga un `id` para poder seleccionarlo.

Sobre el CSS (si usaste)

#	Consejo
1	Separa los estilos en un archivo `.css` externo.
2	Usa clases en lugar de IDs para los estilos.
3	Usa nombres descriptivos para las clases.

Sobre la organización del código

#	Consejo
1	Si tu JavaScript es extenso, sepáralo en un archivo `.js` externo.
2	Usa nombres descriptivos para variables y funciones.
3	Agrega comentarios para explicar la lógica compleja.
4	Mantén una estructura de carpetas clara (`css/` , `js/` , `assets/`).

Resumen del Sprint 2

Tarea	¿Qué debes lograr?	Criterios de evaluación
Tarea 1	Validar el formulario con `validateTaskForm` .	Validación de todos los campos, mensajes de error, archivo JS separado.
Tarea 2	Crear la clase `TaskManager` con sus métodos.	Estructura de tarea, clase y métodos definidos correctamente.
Tarea 3	Conectar el formulario con `addTask` y mostrar tareas.	Tarea en array, visible en pantalla, ID único.

Tarea	¿Qué debes lograr?	Criterios de evaluación
Tarea 4	Agregar botón de eliminar y conectar con `deleteTask`.	Tarjeta eliminada del DOM y del array.
Consejos	Revisar buenas prácticas del HTML del Sprint 1.	No obligatorio, pero recomendado.
